

Here's the complete breakdown — 4 modules for 4 team members across 4 weeks.

4 Modules — Who Owns What

Module 1 — Backend Core & RAG Pipeline

Owner: Team Member 1

Files owned:

- backend/rag_pipeline.py
- backend/db.py
- backend/utils.py

Responsibilities:

- PDF ingestion pipeline (PyPDFLoader → chunking → embedding → ChromaDB)
 - RAG question answering logic
 - Quiz generation logic
 - Vector store management (ChromaDB singleton, similarity search)
 - Session/conversation memory management
 - Prompt engineering (RAG prompt, quiz prompt)
-

Module 2 — API Layer & Data Models

Owner: Team Member 2

Files owned:

- backend/main.py
- backend/models.py
- backend/.env

Responsibilities:

- All FastAPI route handlers (/upload, /ask, /quiz, /health)
 - PDF storage and static file serving (/pdfs/, /documents/list)
 - Request/response validation with Pydantic models
 - CORS middleware configuration
 - Error handling and HTTP status codes
 - API documentation (auto-generated at /docs)
-

Module 3 — Frontend Admin Panel

Owner: Team Member 3

Files owned:

- frontend/app.py → show_login() and show_admin() functions

Responsibilities:

- Login page UI
 - Admin dashboard (Upload tab + Manage tab)
 - PDF upload with multi-file support
 - Inline PDF preview using base64 iframe
 - Delete document functionality
 - Role-based routing logic
 - Auto-start backend subprocess logic
-

Module 4 — Frontend Student Panel

Owner: Team Member 4

Files owned:

- frontend/app.py → show_student() function and all API helper functions

Responsibilities:

- Student chat interface (chat bubbles, message history)
 - Source chunks display toggle
 - PDF viewer tab (dropdown + iframe)
 - Quiz tab (radio buttons, reveal answer)
 - All api_* helper functions (api_ask, api_quiz, api_list_pdfs, etc.)
 - Session management (clear chat, new session ID)
-

4-Week Plan

Week 1 — Setup & Foundation

All members together (Day 1-2):

- Set up Python virtual environment
- Install requirements.txt
- Configure .env with Groq and Google API keys

- Run the existing project once to understand the full flow
- Each member reads their assigned files thoroughly

Member 1 tasks:

- Understand db.py — how ChromaDB stores and retrieves vectors
- Understand rag_pipeline.py — trace the full ingest_pdf() flow
- Test uploading one PDF and verify chunks are created

Member 2 tasks:

- Study all endpoints in main.py
- Test every endpoint using Swagger UI at <http://localhost:8000/docs>
- Understand all Pydantic models in models.py

Member 3 tasks:

- Study the login flow in app.py
- Understand show_admin() — trace what happens when a PDF is uploaded
- Test the admin upload and manage tabs manually

Member 4 tasks:

- Study show_student() — trace a full question → answer flow
- Test the chat, PDF viewer, and quiz tabs as a student
- Understand all api_* helper functions

Week 1 Deliverable: Everyone can run the project, explain their module, and identify one improvement they want to make.

Week 2 — Individual Module Development

Each member works independently on improvements and new features for their module.

Member 1 — RAG Pipeline improvements:

- Add support for uploading .txt and .docx files (not just PDF)
- Improve chunking strategy — try SemanticChunker instead of RecursiveCharacterTextSplitter
- Add a function to delete a document's chunks from ChromaDB when admin deletes a PDF
- Write logging for every step of the pipeline

Member 2 — API improvements:

- Add a GET /documents/{filename}/info endpoint that returns page count and chunk count
- Add input sanitization (strip dangerous characters from filenames)

- Add rate limiting using slowapi library
- Add a GET /stats endpoint showing total documents and total chunks stored

Member 3 — Admin UI improvements:

- Add a progress bar during PDF upload
- Show chunk count and page count per document in the Manage tab
- Add a search/filter box to find documents by name
- Add a confirmation dialog before deleting a document

Member 4 — Student UI improvements:

- Add a typing indicator ("AI is thinking...") while waiting for answer
- Add a "Copy answer" button on each AI response
- Add a score tracker for the quiz (X out of N correct)
- Add a "Download Quiz as PDF" button

Week 2 Deliverable: Each member demos their improvements to the team. Code reviewed by one other member.

Week 3 — Integration & Cross-Module Features

Members work in pairs to build features that span multiple modules.

Pair 1 (Members 1 + 2) — Document Management:

- When admin deletes a PDF via DELETE /documents/{filename}, also delete its chunks from ChromaDB
- Add a POST /reindex endpoint that re-processes all PDFs in pdf_store/ (useful after ChromaDB reset)
- Add document metadata storage (upload date, page count, chunk count) in a simple JSON file

Pair 2 (Members 3 + 4) — User Experience:

- Add a "Recently Asked Questions" section in the student sidebar showing last 5 questions
- Add a topic suggestion feature — after uploading a PDF, auto-suggest 5 topics for quizzes
- Improve the PDF viewer with a page number indicator
- Add a dark mode toggle

All members together (end of Week 3):

- Merge all changes into one working codebase
- Fix any integration bugs

- Test the full flow: Admin uploads → Student asks → Student views PDF → Student takes quiz

Week 3 Deliverable: Fully integrated application with all Week 2 and Week 3 features working together.

Week 4 — Testing, Documentation & Presentation

Member 1 — Testing the RAG pipeline:

- Write test cases: upload a PDF, ask 10 questions, verify answers come from the document
- Test edge cases: empty PDF, corrupted PDF, very large PDF (100+ pages)
- Measure and document average response time

Member 2 — API testing and security:

- Test all endpoints with invalid inputs (empty question, wrong file type, missing fields)
- Verify all error responses return correct HTTP status codes
- Document all endpoints with example requests and responses
- Add .gitignore to exclude .env, chroma_db/, pdf_store/

Member 3 — Admin UI polish:

- Fix any UI bugs found during Week 3 integration
- Add loading states to all buttons
- Write the Admin User Guide (how to upload, manage, preview documents)

Member 4 — Student UI polish:

- Fix any UI bugs found during Week 3 integration
- Improve mobile responsiveness
- Write the Student User Guide (how to ask questions, view PDFs, take quizzes)

All members together (final 2 days):

- Write the final README.md together
- Record a 5-minute demo video
- Prepare a presentation covering: problem → solution → architecture → demo → lessons learned

Week 4 Deliverable: Complete, polished, documented project ready for presentation.

Summary Table

Week Focus	Goal
1 Setup + Understanding	Everyone runs the project and knows their module
2 Individual features	Each member adds 3-4 improvements to their module
3 Integration	Pair work on cross-module features, merge everything
4 Polish + Docs	Testing, bug fixes, documentation, presentation

Module Dependency Map

Module 1 (RAG/DB)

↑ called by

Module 2 (API)

↑ called by

Module 3 (Admin UI) ↔ Module 4 (Student UI)

Member 1 builds the foundation. Member 2 wraps it in HTTP. Members 3 and 4 consume the API. This means Member 1 should finish their Week 2 work first so Members 3 and 4 are not blocked.